

TEMA N° 4



TEMA 4 MEJORA DE RESULTADOS MEDIANTE TÉCNICAS DE PROMPTING



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dominar la estructuración de prompts avanzados** utilizando la técnica de Cadena de Pensamiento (Chain of Thought) para guiar a la Inteligencia Artificial en la resolución de problemas lógicos y de programación complejos dentro de nuestro entorno productivo.
2. **Auditar, validar y depurar de forma interactiva el código fuente** generado por herramientas de IA, identificando vulnerabilidades de seguridad y corrigiendo errores de compilación mediante el análisis crítico de mensajes de error de la consola.
3. **Construir documentación técnica profesional y diseñar discursos de venta (pitches) comerciales** altamente efectivos asistidos por IA, impulsando la creación de emprendimientos tecnológicos sostenibles que respondan a las necesidades reales de la población de Pailón.



PRÁCTICA



¿Alguna vez te ha pasado que entras a una Inteligencia Artificial, le pides un código para resolver una tarea de clase y el resultado que te devuelve es un desastre que no funciona, usa librerías que no existen o calcula cosas completamente ajenas a nuestra realidad? Esto ocurre porque las IA, a pesar de su inmenso poder, no pueden adivinar lo que estamos pensando ni conocen el contexto específico de nuestro entorno si no se lo explicamos de forma correcta.

Para comprender esto de manera práctica, analicemos una situación real en la que se encuentra un estudiante de tercer semestre de la carrera de Sistemas Informáticos en el laboratorio de computación de nuestro nuevo módulo equipado en el **CEA Herman Ortiz Camargo**, ubicado en el Barrio Saó.

El estudiante ha recibido un encargo del señor Donato, propietario de una de las tiendas de abarrotes más grandes del **Barrio Central Fátima** (frente a la plaza principal y la iglesia). Don Donato necesita un pequeño programa en Python para automatizar el cálculo de ventas de sus productos, pero con condiciones muy específicas de la región: algunos productos de la canasta básica están exentos de ciertos recargos, otros tienen descuentos por volumen de venta, y además se debe calcular el Impuesto a las Transacciones (IT) de acuerdo con la normativa boliviana.

El estudiante, entusiasmado por aplicar sus conocimientos, se sentó frente a la computadora del CEA y escribió en la IA el siguiente prompt directo y simple:

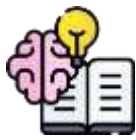
"Haz un programa en Python que calcule las ventas e impuestos de una tienda de abarrotes."

La Inteligencia Artificial le devolvió un script genérico que incluía un impuesto del IVA del 21% (usado en Europa), solicitaba librerías complejas de bases de datos que no estaban instaladas en las computadoras del laboratorio del CEA y, para colmo, cuando el usuario presionaba la tecla "Enter" sin ingresar un número, el programa se cerraba abruptamente mostrando una pantalla llena de letras rojas con errores críticos de ejecución.



El estudiante se encuentra frustrado en el laboratorio del Barrio Saó. El código parece profesional a primera vista, pero es completamente inútil para la tienda de Don Donato en el Barrio Central Fátima. No sabe cómo explicarle a la IA los pasos lógicos que debe seguir, cómo evitar que el programa falle cuando el usuario comete un error de tipeo, ni cómo estructurar las instrucciones para obtener una solución limpia, segura y adaptada a Pailón.

¿Cómo podemos transformar esta frustración en una solución informática real? La respuesta no está en cambiar de IA, sino en aprender a comunicarnos con ella como verdaderos profesionales de los sistemas.



TEORÍA



4.1. IMPLEMENTACIÓN DE PROMPTING DE CADENA DE PENSAMIENTO (CHAIN OF THOUGHT)

El Prompting de Cadena de Pensamiento (o *Chain of Thought*, CoT) es una técnica avanzada que consiste en obligar a la Inteligencia Artificial a desglosar un problema complejo en pasos lógicos secuenciales antes de mostrar la respuesta o el código final. En lugar de pedirle directamente el resultado (lo que a menudo provoca que la IA "alucine" o cometa errores lógicos), le solicitamos que simule el razonamiento humano paso a paso.

Cuando aplicamos CoT, el Técnico en Sistemas Informáticos actúa como un guía pedagógico de la IA. Esta técnica es vital cuando desarrollamos algoritmos que involucran reglas de negocio complejas.

Tomemos como ejemplo la logística de carga en la estación de trenes ubicada en el **Barrio 24 de Septiembre** de Pailón. Supongamos que necesitamos calcular la distribución de peso en los vagones de un tren de carga que transporta soya desde los silos locales. Si le pedimos a la IA el código directamente, podría omitir variables críticas de equilibrio y límites de peso por eje.

Un prompt estructurado bajo la técnica de Cadena de Pensamiento debe configurarse siguiendo este patrón:

1. **Rol:** Definir la identidad de la IA (ej. Experto en algoritmos de optimización).



2. **Contexto:** Explicar el escenario real (Estación de tren del Barrio 24 de Septiembre, Pailón).
3. **Instrucción de Razonamiento:** Indicar explícitamente a la IA que piense paso a paso antes de escribir el código fuente.

Ejemplo de Prompt CoT:

"Actúa como un experto programador en Python. Necesito un script para organizar el peso de los vagones en la estación de trenes del Barrio 24 de Septiembre en Pailón. Para resolver esto, procesa la información de la siguiente manera:

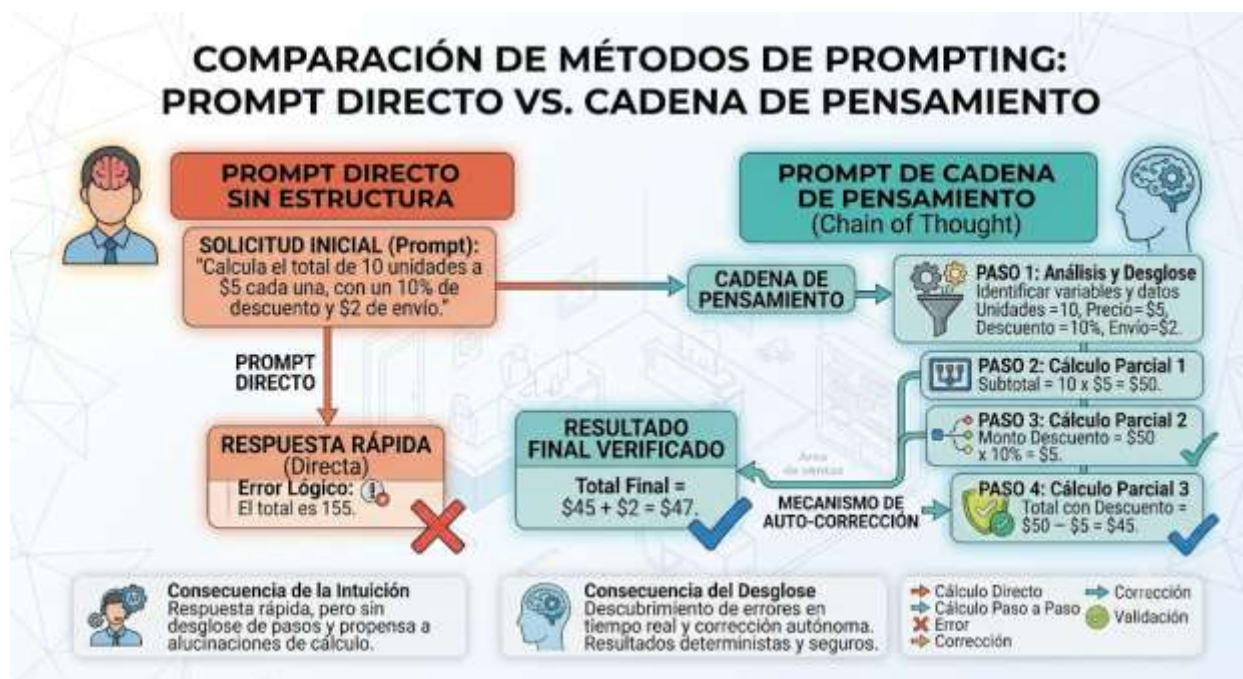
Paso 1: Analiza la lista de pesos de los sacos de soya ingresados.

Paso 2: Explica de forma textual el razonamiento matemático para distribuir los pesos de manera equitativa entre 3 vagones sin superar las 40 toneladas por vagón.

Paso 3: Diseña la lógica de validación para alertar si un vagón está sobrecargado.

Paso 4: Una vez completados los pasos anteriores, escribe el código de Python limpio y optimizado."

Al recibir esta instrucción, la IA estructurará su mente digital, disminuyendo drásticamente la probabilidad de generar código erróneo y permitiendo al Técnico comprender la lógica detrás del software generado.





4.2. VALIDACIÓN Y EVALUACIÓN DE LA SEGURIDAD DEL CÓDIGO GENERADO

Las herramientas de Inteligencia Artificial han sido entrenadas con miles de repositorios de código en internet, muchos de los cuales contienen vulnerabilidades graves de seguridad. Por lo tanto, **un Técnico nunca debe asumir que el código entregado por una IA es 100% seguro o inmune a hackeos.**

En el ámbito de los Sistemas Informáticos, la validación implica revisar exhaustivamente el código en busca de fallas comunes como la inyección de código, el manejo inseguro de credenciales o la falta de validación de datos de entrada.

Imaginemos que estamos desarrollando una plataforma web para que los artesanos del **Barrio Jardines de Pailón** vendan sus productos en línea. Si la IA nos genera un formulario de inicio de sesión o un sistema de búsqueda, es muy probable que cree una consulta SQL directa que sea vulnerable a ataques informáticos.

Veamos un ejemplo de código vulnerable generado comúnmente por una IA frente a la corrección segura que un Técnico debe aplicar tras evaluar la seguridad:

Python

```
# =====
# EJEMPLO DE CÓDIGO VULNERABLE (Generado por IA sin filtros de seguridad)
# =====def
buscar_producto_inseguro(conexion, nombre_usuario):    # Esta línea es
    peligrosa: concatena directamente la entrada del usuario.    # Un atacante en
    Pailón podría escribir "
    artesanía';
DROP TABLE Productos;
--"    # y borraría toda la base de datos del emprendimiento.    cursor =
    conexion.cursor()    query = "
    SELECT * FROM productos WHERE nombre = '' + nombre_usuario + ''
    cursor.execute(query)    return cursor.fetchall()
# =====
# EJEMPLO DE CÓDIGO VALIDADO Y SEGURO (Corregido por el Técnico)
# =====def
buscar_producto_seguro(conexion, nombre_usuario):    cursor = conexion.cursor()
# Usamos consultas parametrizadas (Placeholders '?').    # Esto limpia la
    entrada del usuario y evita cualquier intento de inyección SQL.    query = "
    SELECT * FROM productos WHERE nombre = ?"    cursor.execute(query,
```



```
(nombre_usuario,)) # Pasamos el parámetro de forma segura como una tupla  
return cursor.fetchall() # Retorna los datos validados de la base de datos
```

La validación constante previene la pérdida de datos y protege la confianza de los clientes locales que interactúan con los sistemas desarrollados en nuestro municipio.

4.3. DEPURACIÓN INTERACTIVA: PEGAR ERRORES DEL COMPILADOR PARA OBTENER SOLUCIONES

La depuración (o *debugging*) es el proceso de identificar y solucionar errores en el software. Cuando programamos en la consola del laboratorio del CEA Herman Ortiz Camargo, es normal que surjan excepciones o caídas del sistema. En lugar de adivinar qué está fallando, podemos usar la IA de forma interactiva convirtiéndola en nuestro asistente de soporte técnico.

Para que la IA nos dé la solución exacta a un error de compilación o ejecución, debemos estructurar un prompt de depuración interactiva que contenga cuatro elementos esenciales:

1. El código fuente que está fallando.
2. El mensaje de error completo copiado directamente de la terminal de comandos.
3. El entorno tecnológico de ejecución (versión de Python, Node.js, etc.).
4. Lo que se esperaba que hiciera el programa originalmente.

Supongamos que estamos creando una aplicación para registrar los campeonatos interbarriales en el Coliseo Municipal del **Barrio 27 de Mayo**. El código web encargado de procesar las inscripciones con Node.js se cuelga y muestra un error en la consola.

El prompt de depuración interactiva ideal para pegar en la IA debe ser el siguiente:

"Hola IA, mi código en Node.js está fallando al procesar el registro de equipos para el coliseo del Barrio 27 de Mayo. El código es el siguiente:

[Insertar código aquí]

La consola del sistema me arroja exactamente este error:

TypeError: Cannot read properties of undefined (reading 'nombre_equipo')

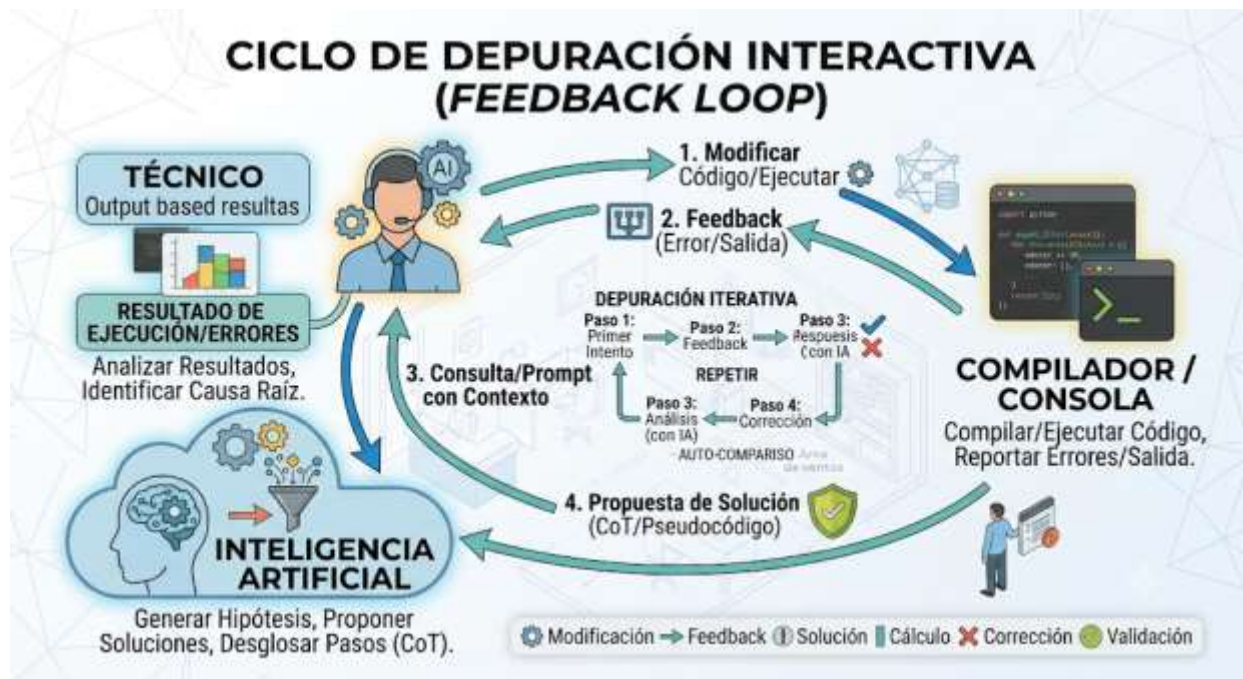


Por favor, analiza el flujo de los datos, explícame por qué la variable llega vacía (undefined) y devuélveme el código corregido con un bloque de control de excepciones (try-catch) para que el servidor no se caiga."

A continuación, se muestra cómo se ve el código problemático y la solución estructurada que la IA proveerá tras la interacción guiada:

JavaScript

```
// =====  
// CÓDIGO ORIGINAL CON ERROR (Se colgaba en la terminal del CEA)  
// =====function  
registrarEquipoColiseo(req, res) {  
  // Si req.body no tiene datos, esta línea destruye la ejecución del programa  
  const nombre = req.body.datos.nombre_equipo;  
  console.log("  
Registrado con éxito en el Barrio 27 de Mayo");  
  res.send("  
Equipo registrado");  
}  
// =====  
// CÓDIGO DEPURADO ASISTIDO POR IA (Estable y seguro)  
// =====function  
registrarEquipoColiseoCorregido(req, res) {  
  try {  
    // Validamos primero si la estructura de la petición contiene los datos  
    // requeridos      if (!req.body || !req.body.datos) {  
      // Enviamos una respuesta HTTP de advertencia en lugar de romper el sistema  
      return res.status(400).send("  
Error: Faltan los datos del equipo");  
    }  
    const nombre = req.body.datos.nombre_equipo;  
    // Ahora la asignación es segura      console.log(`Equipo {  
nombre}  
registrado con éxito en el coliseo.`);  
    res.status(200).send("  
Inscripción completada exitosamente");  
  }  
  catch (error) {  
    // Capturamos cualquier imprevisto técnico y lo registramos sin apagar el  
    // servidor      console.error("  
Error crítico controlado en el sistema del coliseo:", error.message);  
    res.status(500).send("  
Ocurrió un inconveniente interno en el servidor del sistema");  
  }  
}
```



4.4. CREACIÓN DE DOCUMENTACIÓN TÉCNICA (READMEs, MANUALES) ASISTIDA POR IA

Un software excelente sin documentación es un software incompleto. Los Técnicos en Sistemas Informáticos deben asegurar que sus sistemas puedan ser mantenidos, actualizados y comprendidos por otros profesionales o usuarios finales. Redactar archivos README.md (archivos de lectura obligatoria en repositorios de código) o manuales de usuario puede ser tedioso, pero con la IA podemos acelerar este proceso con altos estándares de calidad.

Para documentar con IA, debemos proveerle las funciones principales del código, los requisitos de instalación y el público al que va dirigido.

Imagine que el equipo informático del CEA Herman Ortiz Camargo en el **Barrio Saó** ha culminado el desarrollo de un sistema interno de gestión de notas y asistencia para los estudiantes de la comunidad. El prompt propuesto para la IA sería:

"Actúa como un redactor técnico de software senior. Genera un archivo README.md profesional y claro para el sistema de control académico del CEA Herman Ortiz Camargo del Barrio Saó. El sistema está desarrollado con Python y SQLite. Incluye secciones de: Descripción General, Requisitos



de Software, Pasos detallados para la Instalación en Windows y un ejemplo rápido de uso en consola."

La IA generará un formato estructurado en Markdown listo para ser guardado en el proyecto, tal como se muestra a continuación:

Markdown

```
# Sistema de Gestión Académica - CEA Herman Ortiz Camargo (Pailón)
Este software ha sido desarrollado por los estudiantes de la especialidad de
**Sistemas Informáticos** en el nuevo módulo tecnológico del Barrio Saó. Su
objetivo es automatizar el control de calificaciones y la asistencia del
alumnado del centro.
## 📋 Requisitos del Sistema* Python 3.10 o superior instalado en la computadora
Local.* Gestor de base de datos ligero SQLite3 (integrado nativamente en
Python).
## 📋 Instalación y Despliegue TécnicoSiga estos pasos sencillos en la terminal
de comandos de su computadora:1. **Clonar o descargar el proyecto** en su
directorio local de trabajo.2. **Abrir la consola de comandos** (CMD o Terminal)
y navegar hasta la carpeta raíz del sistema: ``bash cd
C:\SistemasCEA\ControlAcademico
```

1. Ejecutar el script de inicialización para configurar las tablas de la base de datos automáticamente: Bashpython inicializar_bd.py
2. Iniciar la aplicación principal del sistema de gestión: Bashpython app_principal.py

📋 Soporte Técnico

Para reportar fallas en el código o proponer mejoras en los módulos del sistema, por favor contactar con el laboratorio informático del CEA en el Barrio Saó, Pailón.

```
#
## 4.5. Preparación de un "
Pitch" o presentación de ventas usando IAEn el marco del Emprendimiento
Productivo, un Técnico no solo debe saber programar, sino también saber vender
sus soluciones de software a negocios, cooperativas o instituciones del
municipio. Un "
Pitch" es una presentación breve e impactante (de 2 a 3 minutos) diseñada para
convencer a un cliente potencial de adquirir un producto tecnológico o invertir
en una idea de negocio.La estructura clásica de un Pitch exitoso contempla: un
gancho inicial, el planteamiento de un problema real, la presentación de nuestra
solución tecnológica, el modelo de sostenibilidad económica y una llamada a la
acción clara.Visualicemos este escenario: un Técnico del CEA desea vender una
aplicación móvil de entregas rápidas (delivery) a los pequeños comerciantes
gastronómicos ubicados en el **Barrio Ovidio Barberi** y en el **Barrio Las
```



Misiones**. Estos negocios pierden clientes diariamente porque no tienen un sistema coordinado para enviar comida a domicilio dentro de Pailón. Para preparar esta presentación de negocios utilizando Inteligencia Artificial, podemos estructurar un prompt comercial estratégico como este:> *"

Actúa como un asesor experto en oratoria comercial y marketing digital para startups tecnológicas. Necesito crear el guion de un Pitch de ventas de 3 minutos. Mi cliente objetivo son los dueños de negocios de comida de los barrios Ovidio Barberi y Las Misiones en Pailón. Mi software es una app móvil llamada ' PailónExpress

' que conecta mototaxis locales con los restaurantes para hacer entregas en menos de 15 minutos. Estructura el guion incluyendo un título llamativo, una introducción empática con el dolor del comerciante, las ventajas del software y un cierre contundente para que decidan contratar el sistema hoy mismo."*La IA procesará la solicitud estructurando el guion por tiempos de exposición, permitiendo al estudiante practicar su modulación de voz, seguridad expresiva y vocabulario técnico-comercial antes de reunirse con los comerciantes de la zona.



4.6. Prompting de Pocos Disparos (Few-Shot Prompting) para estandarización de datos comerciales de PailónEl *Few-Shot Prompting* (Prompting de pocos disparos) es una técnica avanzada de ingeniería de prompts en la cual le proporcionamos a la Inteligencia Artificial algunos ejemplos resueltos de entradas y salidas antes de pedirle que procese una tarea nueva. Esto le sirve como guía de estilo para comprender con precisión matemática el formato exacto en el que requerimos la información, evitando respuestas discursivas largas. Este enfoque es sumamente útil cuando un Técnico se encuentra limpiando bases de datos desorganizadas aportadas por instituciones o comercios locales. Supongamos que recolectamos direcciones físicas heterogéneas de tiendas en los barrios **San Expedito**, **Oto Waver** y **María Belén**, y necesitamos convertirlas de inmediato a un formato JSON estandarizado para su uso en un mapa



```
digital.**Ejemplo de Prompt con Few-Shot:**> "
```

Actúa como un formateador de datos especializado en Sistemas de Información Geográfica. Voy a darte ejemplos de direcciones escritas de forma informal en Pailón y cómo debes estructurarlas en formato JSON ordenado. Sigue el patrón:*

```
> *Entrada: Tienda frente a la plaza en el barrio María Belén, color azul.*>
*Salida: {
,
barrio': '
María Belén', '
referencia': '
Frente a la plaza principal', '
detalles_visuales': '
Fachada azul
' }
*> > *Entrada: Surtidor saliendo a tres cuadras del coliseo barrio Oto Waver.*>
*Salida: {
,
barrio': '
Oto Waver', '
referencia': '
A tres cuadras del coliseo', '
detalles_visuales': '
Cerca del surtidor
' }
*> > *Entrada: Farmacia en la esquina de la cancha del barrio San Expedito.*>
*Salida:**Al darle este empujón con ejemplos previos, la IA completará la última
línea devolviendo instantáneamente el JSON limpio: `{
,
barrio': '
San Expedito', '
referencia': '
Esquina de la cancha deportiva', '
detalles_visuales': '
No especificado
' }
`. El software consumirá estos datos estandarizados sin arrojar errores lógicos
de lectura de texto.
##
# 4.7. Automatización de pruebas unitarias mediante prompts optimizadosUna
prueba unitaria (Unit Test) es un fragmento de código diseñado para verificar de
manera automática que una función o módulo específico de un programa se comporte
exactamente como se planeó, probando tanto casos normales como datos erróneos.
Escribir pruebas robustas garantiza que, si realizamos modificaciones futuras en
el software, nada del sistema se rompa de manera imprevista.Podemos delegar la
escritura de estas pruebas a la IA mediante prompts que describan detalladamente
las restricciones del negocio. Centrémonos en una solución informática
desarrollada para la junta vecinal del **Barrio San José**: un módulo encargado
de calcular la tarifa de consumo de agua potable comunitaria, donde se aplica
una tarifa básica fija si el consumo es menor a 10 metros cúbicos y un cobro
```



excedente por cada metro adicional. El prompt ideal para generar las pruebas sería: > *"

Actúa como un ingeniero de control de calidad de software (QA). Escribe las pruebas unitarias usando la librería estándar `unittest` de Python para evaluar una función llamada `calcular\_tarifa\_agua(metros\_cubicos)`. La función cobra un mínimo de 30 Bs hasta los 10 metros cúbicos, y 4 Bs por cada metro cúbico extra. Necesito pruebas que validen: 1) Consumo menor al mínimo, 2) Consumo con excedente exacto, y 3) Error si se ingresan valores negativos. *"El código resultante provisto por la IA con comentarios didácticos estructurados línea por línea se presenta a continuación:"*

```
pythonimport unittest
# Función de negocio que el sistema del Barrio San José utilizadef
calcular_tarifa_agua(metros_cubicos):    if metros_cubicos < 0:        # Si el
valor es negativo, lanzamos un error de valor inválido        raise ValueError("
El volumen de agua consumido no puede ser un número negativo")
tarifa_basica = 30 # Costo fijo base en bolivianos        if metros_cubicos <=
10:            return tarifa_basica # Retorna Los 30 Bs mínimos fijados por la junta
else:            metros_extras = metros_cubicos - 10        # Retorna la tarifa base
más el costo de los metros excedentes recalculados        return tarifa_basica +
(metros_extras * 4)
# Clase contenedora de los casos de prueba automatizadosclass
TestSistemaAguaSanJose(unittest.TestCase):    def test_consumo_basico(self):
# Caso 1: Validamos que consumir 8 metros cúbicos cueste exactamente el cobro
básico de 30 Bs        self.assertEqual(calcular_tarifa_agua(8), 30)
def test_consumo_con_excedente(self):        # Caso 2: Validamos que consumir 12
metros cúbicos calcule 30 + (2 * 4) = 38 Bs
self.assertEqual(calcular_tarifa_agua(12), 38)
def
test_valores_negativos_arrojan_error(self):        # Caso 3: Validamos que el
sistema lance la excepción ValueError de forma correcta ante datos erróneos
with self.assertRaises(ValueError):        calcular_tarifa_agua(-5)
# Bloque que permite la ejecución directa de los test en la terminal del CEAI
if __name__ == '__main__':    unittest.main()
```

4.8. EL ROL DEL TÉCNICO EN SISTEMAS COMO "PROMPT ENGINEER" EN EL ECOSISTEMA PRODUCTIVO LOCAL

Con la llegada de modelos de lenguaje avanzados, la programación tradicional ha evolucionado. Hoy en día, escribir código desde cero ya no es la única tarea del informático; el rol del **Técnico en Sistemas Informáticos** se ha transformado en el de un orquestador tecnológico, actuando como un "Prompt Engineer" (Ingeniero de Prompts) capacitado.

Esto significa que el Técnico se convierte en el puente de comunicación directo entre las necesidades comerciales del municipio de Pailón y las herramientas globales de IA. Diseñar prompts no es "chatear" por entretenimiento; requiere un entendimiento profundo del flujo de control de datos, algoritmos, arquitectura de sistemas y seguridad web.



En nuestro entorno, el Técnico del CEA Herman Ortiz Camargo aporta valor al tejido social aplicando estas destrezas para digitalizar peluquerías en el **Barrio San Antonio**, optimizar la logística de transporte en las terminales de buses o implementar sistemas de cobros en los mercados locales. Un prompt optimizado ahorra horas de desarrollo y reduce costos operativos, convirtiendo al Técnico en un pilar del emprendimiento productivo e innovación local.

CIERRE TEÓRICO OBLIGATORIO

□ Mitos vs. Realidades

Mitos sobre el uso de Inteligencia Artificial en Sistemas	Realidades respaldadas por la Ingeniería Informática
La IA genera código perfecto y libre de fallas, por lo que se puede copiar, pegar y desplegar directamente en cualquier empresa.	La IA crea código propenso a errores lógicos, librerías obsoletas y serias brechas de seguridad; el Técnico siempre debe auditarlo.
El uso de técnicas de prompting reemplaza la necesidad de estudiar programación o lógica de desarrollo de sistemas informáticos.	Para diseñar prompts efectivos y depurar errores es obligatorio dominar las bases de la lógica, algoritmos y lenguajes de programación.

□ Glosario de Términos

1. **Prompt:** Instrucción estructurada en lenguaje natural (texto) que se le proporciona a un modelo de IA para dirigir y moldear la respuesta que genera.
2. **Chain of Thought (Cadena de Pensamiento):** Técnica de prompting que instruye a la IA a resolver problemas desglosándolos explícitamente en una secuencia lógica de pasos intermedios.
3. **Inyección SQL:** Ataque cibernético malicioso que consiste en introducir comandos maliciosos disfrazados de texto común dentro de formularios web para manipular las bases de datos de un sistema.
4. **Refactorización:** Proceso técnico que modifica la estructura interna del código fuente de un programa para hacerlo más limpio, eficiente y seguro, sin alterar en absoluto su funcionamiento externo original.



5. **Prueba Unitaria (Unit Test):** Código automatizado encargado de aislar y validar de forma matemática el correcto funcionamiento de una sección específica e individual de un software.

□ **Ponte a Prueba**

1. ¿Cuál es el objetivo principal al aplicar la técnica de prompting de Cadena de Pensamiento (Chain of Thought)?

1. A) Hacer que el código corra más rápido en servidores web antiguos.
2. B) Forzar a la IA a desglosar el razonamiento del problema paso a paso antes de entregar el código fuente final.
3. C) Traducir automáticamente un programa de Python a lenguaje JavaScript.

2. Si una Inteligencia Artificial nos genera un bloque de código para interactuar con bases de datos, ¿cuál debe ser la postura del Técnico respecto a la seguridad?

1. A) Confiar plenamente e instalarlo de inmediato en el servidor del cliente local.
2. B) Modificar las variables visuales para cambiar los colores de la aplicación únicamente.
3. C) Evaluar y validar el código para descartar vulnerabilidades críticas como la concatenación directa de datos o inyecciones SQL.

3. ¿Qué datos mínimos se deben recopilar e ingresar a una IA en una sesión de depuración interactiva efectiva?

1. A) El nombre del sistema, el color de la interfaz y la fecha actual.
2. B) El código con la falla, el mensaje de error completo de la terminal, el entorno tecnológico y el comportamiento esperado.
3. C) La historia del negocio y los datos personales de los clientes del emprendimiento.

Respuestas correctas: 1-B, 2-C, 3-B.



VALORACIÓN



El uso de la Inteligencia Artificial en el desarrollo de sistemas de software trae consigo un profundo impacto en múltiples esferas que todo Técnico debe sopesar de manera ética y consciente.

El Impacto Ético y de Privacidad: Cuando utilizamos plataformas de IA en línea para programar aplicaciones para negocios reales de Pailón (como los comercios formales del Barrio Central Fátima), es común que los desarrolladores cometan el error de pegar bases de datos completas que contienen nombres, números de cédula de identidad o historiales financieros de los clientes locales para que la IA los ordene. Al hacer esto en plataformas públicas, estamos entregando información confidencial a servidores externos, violando flagrantemente el derecho a la privacidad de nuestros ciudadanos. La ética profesional nos obliga a usar datos ficticios o "anonimizados" durante la fase de desarrollo asistido.

El Impacto Social y la Brecha Digital: La incorporación de la IA en los emprendimientos productivos de nuestro municipio no debe convertirse en una herramienta de exclusión, sino de democratización. Un Técnico con conciencia social debe enfocar sus habilidades para simplificar los procesos de los pequeños productores agrícolas y comerciales, reduciendo la brecha digital en lugar de ampliarla, permitiendo que las tecnologías modernas fortalezcan la economía de los barrios populares de Pailón.

El Impacto Medioambiental y el E-Waste: El procesamiento de millones de prompts complejos en supercomputadoras globales consume cantidades colosales de energía eléctrica y requiere sistemas de refrigeración basados en millones de litros de agua potable diarios, acelerando la huella de carbono global. A nivel local, el acelerado ritmo de actualización tecnológica puede provocar que las computadoras queden obsoletas rápidamente, convirtiéndose en basura electrónica (e-waste). En el CEA Herman Ortiz Camargo valoramos el cuidado del medio ambiente; por lo tanto, optimizar los prompts para evitar iteraciones innecesarias y dar mantenimiento preventivo al equipamiento de nuestro nuevo módulo es un deber ambiental ineludible para preservar los recursos de nuestra región.



PRODUCCIÓN



LABORATORIO PRÁCTICO INTEGRADOR

Título del Desafío: Desarrollo, Auditoría de Seguridad y Pitch de un Sistema de Control de Cuotas de Socios para la Asociación de Transportistas en el Barrio 24 de Septiembre, Pailón.

Objetivo General: El estudiante implementará de punta a cabo las técnicas de prompting analizadas en este Tema 4 para codificar en Python, validar la seguridad, documentar con archivos README y preparar la venta de un sistema informático real adaptado a las necesidades de transporte de nuestra provincia.

Instrucciones paso a paso para el desarrollo en las computadoras del CEA:

Paso 1: Configuración de la IA y Prompting CoT (Cadena de Pensamiento)

Abra el navegador web en las computadoras del laboratorio de computación del módulo del CEA del Barrio Saó e ingrese a la herramienta de Inteligencia Artificial disponible. Escriba y ejecute un prompt basado en la Cadena de Pensamiento para estructurar un script de Python que registre las cuotas semanales de los transportistas de la terminal del Barrio 24 de Septiembre (Monto fijo: 50 Bs). El prompt debe obligar a la IA a explicar primero la lógica de las condiciones matemáticas (pagos puntuales, retrasos con multa del 10% si se paga fuera de fecha) antes de arrojar el código fuente.

Paso 2: Auditoría y Despliegue del Código Fuente

Copie el código generado por la IA en su entorno de desarrollo local (VS Code o IDLE Python en la computadora del CEA). Revise línea por línea que la lógica matemática coincida con las multas del entorno local de Pailón. Inserte manualmente comentarios en español en cada función explicando el control del flujo del programa. Asegúrese de implementar bloques de captura de errores try-catch (try-except en Python) para evitar que, si un usuario digita letras en lugar de números en la consola, el script se congele.

Paso 3: Depuración Interactiva Provocada



Fuerce un error de forma intencional en su código eliminando los dos puntos (:) de una sentencia condicional if o mandando a llamar a una función que no exista. Ejecute el programa en la terminal y copie textualmente las líneas de error rojas arrojadas por el compilador de Python. Diseñe un prompt interactivo pegando su código dañado, el error de la terminal y solicítele a la IA que diagnostique la falla estructural y le devuelva la solución limpia. Documente este proceso en su cuaderno técnico de laboratorio.

Paso 4: Generación Asistida del Manual Técnico README

Solicite a la Inteligencia Artificial la redacción de un archivo Markdown (README.md) para el sistema de transportistas. El archivo debe detallar el nombre del software ("PailónTrans-Control"), las librerías necesarias y los comandos exactos para ejecutarlo en entornos Windows dentro de las oficinas de la asociación en el Barrio 24 de Septiembre. Guarde este texto con el nombre de archivo README.md dentro de la carpeta de su software en la computadora.

Paso 5: Diseño del Pitch Comercial para la Asociación de Transportistas

Para culminar el laboratorio, pídale a la IA que actúe como un especialista en marketing tecnológico y redacte el guion de un Pitch de Ventas de 2 minutos dirigido a la junta directiva de transportistas de Pailón. El discurso debe enfatizar cómo el sistema evitará la pérdida de dinero, agilizará los reportes y se adaptará a celulares y computadoras de escritorio. El estudiante deberá realizar una simulación de oratoria frente a sus compañeros de clase en el aula del CEA emulando una reunión formal de negocios.

RECURSOS ADICIONALES

1. **Documentación Oficial de Python (Módulo unittest):** Guía teórica indispensable para profundizar en la sintaxis y creación de casos de pruebas unitarias automatizadas para software confiable.
2. **Guía de Markdown de GitHub:** Enlace educativo recomendado para aprender a formatear texto enriquecido, tablas, listas e hipervínculos para crear archivos README de estándar internacional.



3. **OWASP Top 10 (Fundación de Seguridad en Aplicaciones Web):** Portal de referencia global para estudiar las vulnerabilidades de software más comunes en la actualidad y aprender metodologías de validación de seguridad contra ciberataques.